The Need For Data Quality Validation and Testing

In this train, we'll discover how to ensure your data is clean and reliable through data quality validation and testing. We'll explore the fundamentals of safeguarding data integrity, including how data quality validation acts as an assurance of incoming data and how data testing serves as integration testing.

Learning Objectives

By the end of this train, you should be able to:

- Data quality validation as an assurance of incoming data.
- Data testing as integration testing.

Understand The Need For Data Quality Validation and Testing

Data quality is a crucial part of data engineering. After ingesting data into your system, the next important thing is to ensure that the data is of sufficient quality to allow for high-quality and robust science and analytics. Imagine you build the most efficient, robust, and elegant data pipelines to process your data. All this work would be for nothing if your data is not an accurate representation of real-world events. Therefore, you must prioritise data quality from the start of your project. There are a few cases that may occur when your data quality is not up to scratch:

- Your code checks the input data with expectations based on data that you know is correct. If the data is not up to scratch, then you initiate a plan B. This could include re-running the API to retrieve the missing data or alerting users and data engineers to the problem.
- The code runs into an error while processing data and crashes. This may occur when your data pipelines expect a specific schema or data type.

- Lastly, your data pipelines could continue to run regardless of the poor data quality, producing outputs that look correct but are based on substandard data.

Of these three cases, the first one is the best, as we are at least being alerted to changes in data quality. This is the only case where we have data quality checking in place. The second case is not the end of the world – with a failure you are at least aware that something has failed and can look for a solution or re-run your data pipeline. The last case is very dangerous as you could remain unaware of these data quality issues for months. Today's applications and businesses, in general, are gravitating towards data-driven products and decision-making. As you know, there are huge amounts of data available and this is only increasing. Businesses are becoming more dependent on using data and making informed decisions. Therefore, substandard data is not an option, as it can lead to incorrect modelling or analytics. In the sections below, we are going to take a deeper look at data quality and dive into some of the use cases for data quality checks. We'll also introduce six dimensions of data quality that can be evaluated.

Data Quality Validation As An Assurance of Incoming Data

Data quality validation means making sure that the incoming data is of sufficient quality. Here we will expand on how to validate the quality of the incoming data and show which mitigation should take place when there is a change in data quality. Two important factors need to be monitored here. The first is the change in data quality, such as missing values or duplicates. The second is a change in data distributions where values are outside the expected range. Let's take a look at how to approach data quality validation. The first step in tracking data quality is deciding on the data requirements:

Defining the schema and acceptable data types of a dataset ensures that changes to either of these do not negatively affect downstream processes, such as modelling and analysis.

Data requirements can also include the six dimensions of data quality, which can direct decisions around data requirements.

It could also include defining the bounds of what qualifies as a “normal” value, for example, one standard deviation from the mean historical value.

Once you have decided on the data requirements, you must automatically check that these requirements are met. This may look like a data quality pipeline running once a day or running every time new data is ingested which tests for the defined data quality requirements. This process could also include an alerting system that lets you (as the data engineer) know about violations in the data requirements. Data quality is likely to vary depending on the source system. It cannot necessarily be changed, but you can

ensure that the correct systems are in place to continuously mitigate the risks associated with substandard data. This may be in the form of a data quality rating displayed on the front-end of your application so that users know when data are unreliable. Another option is creating a system that checks incoming data based on requirements and activates a recovery pipeline that re-ingests data if the faults are due to the ingestion step. These mitigation options aim to ensure awareness around data quality and that you are not falling into the third case mentioned above, where your data pipelines continue to run regardless of substandard data.

> **Remember that mitigation plans for poor data quality are highly dependent on the requirements.** Therefore, you should treat each new dataset as a distinct problem to ensure no additional faults come into play.

Data Testing As Integration Testing

Testing is the second use for data quality. Here we use data quality as a method for integration testing. Thus, monitoring data quality through a processing pipeline on a mock dataset can tell us if the pipelines are performing as they should. This approach takes on a more rigorous design than ensuring that incoming data is accurate. Once we are sure that incoming data is up to the correct standard, there are more places where data testing and validation can be included. Following the logic of ETL, once we have extracted the data and ensured its quality, there are likely many transformations that need to be built into the data pipelines based on various business requirements.

Consider The Following Example:

A business receives data every hour for the number of times a pump within a water distribution network has started in its lifetime. This metric is useful as it can help the business determine how close a pump is to the end of its life (water pumps only effectively pump water for a set amount of pump starts). This data can provide a further benefit by calculating the number of times a pump has started in a day, which can alert the business to where pressure on their system is. To calculate the number of times a pump has started in a day, it is necessary to keep track of the previous days' cumulative number of starts and use this as a baseline for that day's number of starts. This is where data testing can be taken to a new level. If we know the method of calculating the number of starts per day, and we have a mock dataset with a known number of starts that occurred, we can create a testing script to validate that the calculation is performing as expected. This involves providing the function that performs the calculation with known inputs and outputs, and testing if it passes or fails

the test, so we can know with certainty that our function performs correctly. It is very similar to how we would test general software functionality.

> **This is an example of integration testing** for one function or transformation, but these types of checks should ideally be made at every point that your data changes form.By employing this type of testing, you can safely make changes and refactor your code while still having the tests in place to validate that you haven't broken anything.

Conclusion

Data quality validation and testing are essential for building reliable, trustworthy data pipelines. By proactively checking incoming data and applying integration tests throughout your pipeline, you can prevent silent failures and ensure your data-driven decisions are based on accurate, high-quality information.

▼Key Takeaways ✖

- **Takeaway 1:** Always validate incoming data to catch issues early and avoid downstream problems.
- **Takeaway 2:** Monitor for changes in schema, data types, and data distribution to maintain integrity.
- **Takeaway 3:** Use data testing as a form of integration testing, especially after transformations.
- **Takeaway 4:** Build automated data quality checks with alerts to stay informed of issues.
- **Takeaway 5:** Treat every new dataset uniquely—custom checks and mitigation strategies are key.